WEEKS 14–15 · LABORATORY ACTIVITY

Secure It, Then Prove It

Real JWT authentication + an automated test suite — on the SecureNotes API

Course	Units	Work in	Est. time	Submit
Web Systems and Technologies	1 (Laboratory)	Pairs	3–4 hours	GitHub PR links

The scenario. In Week 13 you hardened **SecureNotes**. But its login is still a toy — passwords in plain text, a "token" that's just the username. In this lab you do two things a real team always does together: first you **build the security properly** (hashed passwords + JSON Web Tokens), then you **write the tests** that prove it works and can't quietly break later. Part A builds it; Part B proves it.

What you'll practice

Part A — build it

- bcrypt password hashing (A04)
- signing & verifying a JWT in FastAPI (A07)
- a dependency that fails closed on a bad token
- ownership & role checks (A01)

Part B — prove it

- pytest + FastAPI TestClient
- a fixture for a clean state each test
- happy-path and error-path API tests
- security tests that lock in the OWASP rules

Setup (10 minutes)

```
# inside the student folder
python -m venv venv
source venv/bin/activate   # Win:
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload   # the app
pytest -q                       # 2 pass, rest
TODO (red)
```

Accounts

- `admin / admin123` (admin)
- `alice / alicepass` — owns note 1
- `bob / bobpass` — owns note 2

Keep the bcrypt pin (`bcrypt==4.0.1`) — newer bcrypt breaks passlib.

Docs at `http://127.0.0.1:8000/docs` .

Part A — Make the security real (~75 min)

Edit only inside the `>>> TASK n ... <<< END TASK n` markers in `app/main.py` . Work top to bottom.

Task	OWASP	What to do
1	A04	**Hash passwords.** `hash_password` returns a bcrypt hash; `verify_password` checks one.
2	A07	**Issue & verify a JWT.** `create_token` signs a JWT with `sub` + `exp` ; `read_token` decodes it, returns the username or `None` .
3	A07	**Fail closed.** `get_current_user` raises `401` on a missing/forged/expired token or unknown user.
4	A01	**Ownership.** `get_note` returns `403` when the note isn't the caller's.
5	A01	**Role.** `/admin/users` requires the admin role and returns only `id, username, is_admin` .

Everything here was in the Week 14 lecture — `pwd.hash` / `pwd.verify` , `jwt.encode` / `jwt.decode` , a `Depends` dependency, and an `if` check. No new libraries needed.

Part B — Write the tests that prove it (~75 min)

Now prove Part A actually works. In `tests/conftest.py` you're given a `client` fixture (fresh DB per test) and helpers `login(client, user, pw)` and `auth(token)`. Replace each `assert False, "TODO..."` with a real test.

API — `test_api.py`

- ☐ register, then login returns a JWT
- ☐ wrong password returns `401`
- ☐ create a note, then see it listed

Security — `test_security.py`

- ☐ bad token rejected (`401`) A07
- ☐ can't read another's note (`403`) A01
- ☐ can read your own note (`200`)
- ☐ same error: bad user vs bad password A07
- ☐ non-admin can't list users (`403`) A01
- ☐ admin list has no `password` field A04

```
# a good test — Arrange, Act, Assert
def test_user_cannot_read_another_users_note(client):
    alice = login(client, "alice", "alicepass")          # Arrange
    res = client.get("/notes/2", headers=auth(alice))    # Act (note 2 is Bob's)
    assert res.status_code == 403                        # Assert — A01
```

> **The two halves check each other.** If your Part A is correct, your Part B tests go green. If a test stays red, either the test is wrong or you missed something in Part A — both worth finding now. Goal: **all 11 tests pass**, then run `pytest --cov=app` . Coverage is a hint, not a grade — every test must have a real `assert` .

Part C — Peer review on GitHub (~30 min)

```
git init
git add . && git commit -m "Starter: SecureNotes (toy auth + test stubs)"
git branch -M main
git remote add origin https://github.com/<you>/securenotes-lab.git
git push -u origin main
git checkout -b secure-and-test
# ...do Part A, then Part B...
git add . && git commit -m "Real JWT auth + full test suite"
git push -u origin secure-and-test
```

Open a pull request (`secure-and-test → main`), list what you did, and share the link. As a reviewer, pull the branch, run the app *and* `pytest` , and leave **line comments** on the diff — then **Approve** or **Request changes**. Keep `venv/` out of the repo (a `.gitignore` is included).

Deliverables & rubric

Submit (via the LMS):

1. GitHub **repo** link (`secure-and-test` branch).
2. Your **pull request** link.
3. The **PR you reviewed** (comments + verdict).
4. Completed `TASKS.md` (checklist + reflection).

These tests become the safety net that runs automatically in Week 17 (CI/CD).

Criteria	Pts
Part A — 5 auth tasks work	**60**
Part B — 9 tests pass	**30**
Tests independent / real asserts	**5**
Peer review + reflection	**5**